AI Engineer Assignment

Overview

Build a multi-agent system that researches a technical topic and produces a structured analysis report. This assignment tests your ability to design agent workflows, select appropriate tools/frameworks, and implement a production-quality solution.

Requirements

Core Functionality

Your system must:

1. **Research Agent**
    - Accept a technical topic as input
    - Search multiple sources (web, documentation, papers)
    - Extract and validate information
    - Handle rate limits and failures gracefully
2. **Analysis Agent**
    - Synthesize research findings
    - Validate research quality and completeness
    - Identify gaps or contradictions
    - Suggest improvements or request additional research
    - Generate insights and recommendations
    - Maintain context across agent interactions
3. **Report Generation Agent**
    - Structure findings into a coherent report
    - Include citations and sources
    - Generate executive summary
    - Export to markdown/PDF

Technical Requirements

- **Multi-agent orchestration** with clear workflow
- **Tool/API integration** (search, document processing, etc.)
- **State management** across agent interactions
- **Error handling and retry logic**
- **Observability** (logging, tracing)
- **Configuration management** (API keys, parameters)
- **Feedback loops** between agents (e.g., Analysis → Research for gaps)
- **Iteration control** (max retries, convergence criteria)

Deliverables

1. **Working Implementation**
    - Full source code with proper structure
    - Environment setup instructions and example run with output
2. **Technical Decision Document**
    - See section below

Evaluation Criteria

Implementation (40%)

- Code quality and organization
- Proper error handling
- Working end-to-end flow
- Documentation and setup clarity

Architecture & Design (30%)

- Agent separation of concerns
- Workflow design
- State management approach
- Extensibility

Technical Decisions (20%)

- Framework/tool justification
- Understanding of trade-offs
- Awareness of alternatives
- Production considerations

Problem Solving (10%)

- Handling edge cases
- Error recovery strategies
- Resource management

Constraints

- **Time limit**: 4-6 hours of development time
- **Budget**: Use free tiers
- **Language**: Python preferred (or specify your choice)
- **Testing**: At least one successful end-to-end run with example output

Technical Decision Document

Answer these questions in your documentation:

Architecture & Framework Decisions

- **Framework Selection**: Why did you choose your multi-agent framework (LangGraph, CrewAI, AutoGen, custom, etc.)? What alternatives did you consider? What limitations did you encounter?
- **LLM Selection**: Which LLM(s) did you use and why? Did you use different models for different agents?
- **State Management**: How do agents share context? Why this approach?
- **Orchestration Pattern**: How do agents interact? Sequential, parallel, or dynamic routing?

Implementation Details

- **Tool Integration**: How did you implement tool calling? What tools/APIs did you integrate?
- **Agent Differentiation**: What are the key differences between your agents in terms of prompting strategies, tool access, and decision-making authority?
- **Error Handling**: How does your system handle agent failures? Describe your retry strategies, fallback mechanisms, and state recovery approach.

Optimization & Production Readiness

- **Token/Cost Optimization**: What's your approach to managing costs?
- **Scalability**: What would you change for production at scale?
- **Convergence & Control**: How do you prevent infinite loops? What are your exit conditions and iteration limits?
- **Trade-offs & Limitations**: Known limitations of your implementation? Edge cases not handled?

Good luck! We're excited to see your approach to multi-agent system design.